

Framework Design Justification

When I built snap-framework, I followed five design principles called SOLID. These principles are guidelines that help make code easier to understand, maintain, and expand over time. Here is how I applied each one.

Single Responsibility — "One job per class." Every class in my framework does only one thing. For example, the Router class only figures out which page the user is trying to visit. The Request class only holds the data coming from the browser. The Validator class only checks if form inputs are correct. I kept them separate so that if something breaks or needs to change, I only have to touch one file instead of digging through many.

Open/Closed — "Add new things without breaking old ones." I designed parts of the framework so new features can be added without editing existing code. For example, my database system supports both MySQL and SQLite through a DatabaseDriver interface. Each database type — MySQLDriver and SQLiteDriver — has its own class. If I ever needed to add PostgreSQL support, I would just create a new class. I would not need to touch the Connection class at all. The same idea applies to repositories. TaskRepository extends EntityRepository and adds a findByStatus() method without changing any of the shared logic.

Liskov Substitution — "Replacements should work the same way." This principle is about making sure that when one class can stand in for another, everything still works correctly. In my framework, MySQLDriver and SQLiteDriver can be swapped in public/index.php without anything else breaking. The TaskController calls methods like find() and save() on a TaskRepositoryInterface — it does not care which specific class is behind it. As long as the class follows the interface contract, the controller works the same.

Interface Segregation — "Don't force classes to do things they don't need." Instead of making one big interface that covers everything, I split data access into two small focused interfaces. Findable only covers reading data — find() and all(). Persistable only covers writing data — save() and delete(). This way, a class that only needs to read data is not forced to deal with save or delete methods it will never use. TaskRepositoryInterface simply combines both, keeping things clean and specific.

Dependency Inversion — "Classes should ask for what they need, not create it themselves." Instead of a controller creating its own database connection or repository, it just declares what it needs in the constructor and the framework provides it. TaskController asks for a TaskRepositoryInterface and an Engine — it never calls new TaskRepository() itself. My Container class handles all of that automatically. It reads the constructor, figures out what each class needs, and builds everything in the right order. This means I can easily swap out one implementation for another without changing the controller at all. Overall, following SOLID helped me build a framework where each piece has a clear purpose, changes in one area do not break other areas, and the system can grow without becoming messy.